

Validador Formal em Três Níveis

Projeto Prático — Modelagem Computacional
Hierarquia de Chomsky: LR ■ LLC ■ R

Nível	Linguagem	Modelo	Aplicação
LR	Regular	DFA (15 estados)	CPF — ddd.ddd.ddd-dd
LLC	Livre de Contexto	PDA (pilha explícita)	Delimitadores balanceados
R	Recursiva	Máquina de Turing	$L = \{w\#w \mid w \in \{0,1\}^*\}$

1. Introdução e Contexto Aplicado

Toda vez que um software decide se uma entrada é válida, ele está reconhecendo uma linguagem formal. Validadores de formulário (CPF, e-mail, IPv4), parsers de protocolo (HTTP, TCP), verificadores de logs e mecanismos de detecção de intrusão fazem isso continuamente.

Esses validadores diferem fundamentalmente no poder computacional necessário para resolver cada problema. O presente projeto constrói três reconhecedores formais, um para cada nível relevante da Hierarquia de Chomsky, demonstrando empiricamente a relação de inclusão estrita LR ■ LLC ■ R.

O tema unificador é **validação de cadastro e integridade de dados**: validação de formato de CPF (Regular), verificação de delimitadores balanceados em expressões simbólicas (Livre de Contexto) e verificação de duplicação exata de cadeia binária separada por # (Recursiva).

2. Linguagem Regular — DFA para CPF

2.1 Descrição

A linguagem reconhece cadeias no formato **ddd.ddd.ddd-dd**, onde cada *d* representa um dígito decimal (0–9). O formato consiste em três blocos de três dígitos separados por pontos, seguidos de um hífen e dois dígitos finais. Apenas o formato textual é validado — os dígitos verificadores do CPF não são computados.

2.2 Definição Formal

$$\Sigma = \{0, 1, \dots, 9, '.', '-'\}$$

$$L = \{ w \in \Sigma^* \mid w = d_1d_2d_3.d_4d_5d_6.d_7d_8d_9-d_{10}d_{11}, \text{ com cada } d_i \in \{0, \dots, 9\} \}$$

2.3 Modelo Computacional — DFA

O reconhecedor é um Autômato Finito Determinístico (DFA) com **15 estados** (q0–q14) e um estado morto (qDEAD). O estado inicial é q0 e o único estado de aceitação é q14, atingido exatamente após a leitura dos 14 símbolos que compõem um CPF válido.

Tabela de Transição (δ) — DFA CPF:

Estado	Símbolo lido	Próximo estado	Significado
q0	0–9	q1	1º dígito do bloco 1
q1	0–9	q2	2º dígito do bloco 1
q2	0–9	q3	3º dígito do bloco 1
q3	'.'	q4	Ponto separador 1
q4–q6	0–9	q5–q7	Bloco 2 (3 dígitos)
q7	'.'	q8	Ponto separador 2
q8–q10	0–9	q9–q11	Bloco 3 (3 dígitos)
q11	'-'	q12	Hífen
q12	0–9	q13	1º dígito final
q13	0–9	q14 ✓	2º dígito final — ACEITA

q14	qualquer	qDEAD	Símbolo extra — REJEITA
qDEAD	qualquer	qDEAD	Estado absorvente

DFA — CPF (ddd.ddd.ddd-dd)

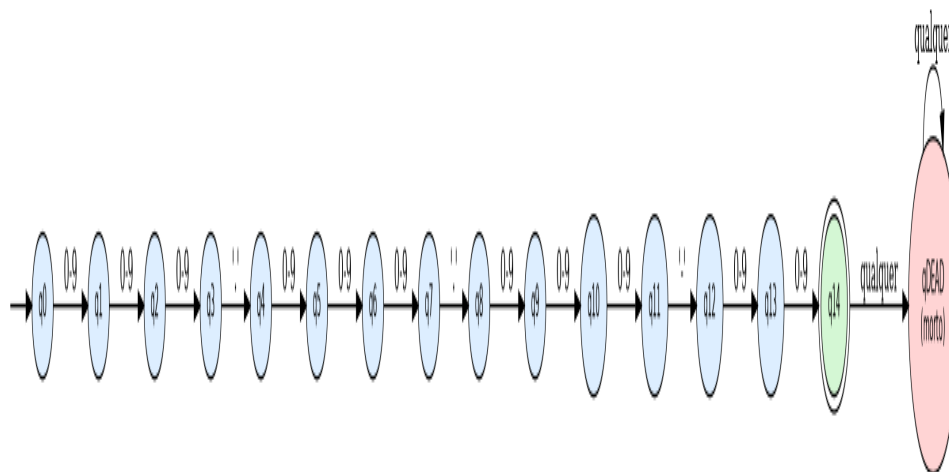


Figura 1 — Diagrama do DFA para CPF

2.4 Exemplos

Cadeia	Resultado	Passos
123.456.789-00	✓ ACEITA	14
000.000.000-00	✓ ACEITA	14
999.999.999-99	✓ ACEITA	14
12.34.56-78	✗ REJEITA (bloco com 2 dígitos)	11
123.456.789-001	✗ REJEITA (dígito extra)	15
12345678900	✗ REJEITA (sem separadores)	11

2.5 Execução Passo a Passo — '123.456.789-00' (aceita)

q0 → '1' → q1 | q1 → '2' → q2 | q2 → '3' → q3 | q3 → '.' → q4 | q4 → '4' → q5 | q5 → '5' → q6 | q6 → '6' → q7 | q7 → '.' → q8 | q8 → '7' → q9 | q9 → '8' → q10 | q10 → '9' → q11 | q11 → '-' → q12 | q12 → '0' → q13 | q13 → '0' → q14 ✓ [14 passos]

2.6 Execução Passo a Passo — '12.34.56-78' (rejeita)

q0 → '1' → q1 | q1 → '2' → q2 | q2 → '.' → qDEAD [3 passos úteis + 8 absorvidos = 11 passos; terceiro símbolo deveria ser dígito]

3. Linguagem Livre de Contexto — PDA para Balanceamento

3.1 Descrição

A linguagem reconhece expressões simbólicas em que todos os delimitadores de abertura (parênteses, colchetes e chaves) possuem um delimitador de fechamento correspondente, na ordem correta e sem sobreposição. Caracteres alfanuméricos e operadores são neutros e não afetam o balanceamento.

3.2 Definição Formal

$$\Sigma = \{ (,), [,], \{, \}, a-z, A-Z, 0-9, +, -, *, /, ^, \dots \}$$
$$L = \{ w \in \Sigma^* \mid \text{todo abre-delimitador em } w \text{ tem um fecha-delimitador correspondente, respeitando a ordem de aninhamento} \}$$

GLC equivalente (simplificada):

$$S \rightarrow \varepsilon \mid SS \mid '(' S ')' \mid '[' S ']' \mid '\{ ' S '\}'$$

3.3 Modelo Computacional — PDA

O reconhecedor é um Autômato com Pilha (PDA) determinístico. A pilha contém o marcador de fundo '\$' e os delimitadores de abertura ainda não fechados. O estado inicial e de aceitação é q0; a aceitação ocorre quando, ao final da entrada, a pilha contém apenas '\$'.

Principais Transições do PDA:

Estado	Símbolo lido	Topo da pilha	→ Estado	Ação na pilha
q0	'('	*	q0	push '('
q0	'['	*	q0	push '['
q0	'{'	*	q0	push '{'
q0	')'	'('	q0	pop
q0	']'	'['	q0	pop
q0	'}'	'{'	q0	pop
q0	neutro	*	q0	sem mudança
q0	']'/'}'/';...	topo errado	qDEAD	erro de balanceamento
qDEAD	qualquer	*	qDEAD	absorvente

PDA — Balanceamento de Delimitadores

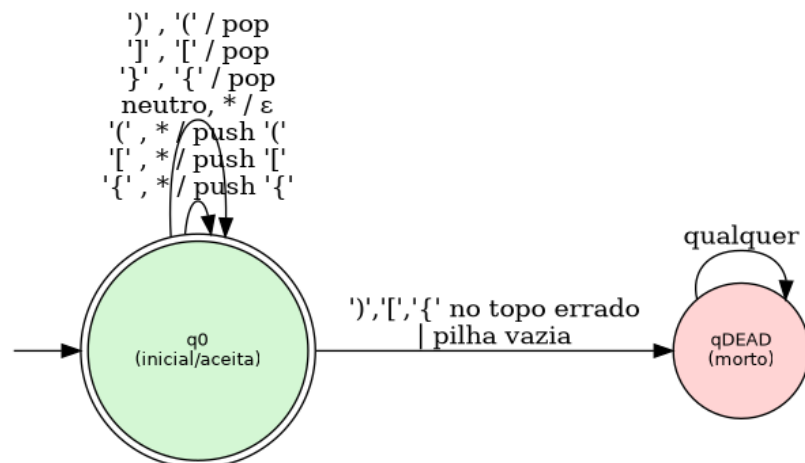


Figura 2 — Diagrama do PDA para balanceamento

3.4 Exemplos

Cadeia	Resultado	Passos
$((x+y)*z)$	✓ ACEITA	9
$\{[a+b]*(c-d)\}$	✓ ACEITA	13
$(\{\}\})$	✓ ACEITA	6
$((a+b)$	✗ REJEITA (parêntese não fechado)	6
$([])$	✗ REJEITA (fechamento cruzado)	3
$)$	✗ REJEITA (fecha sem abrir)	1

3.5 Execução Passo a Passo — ' $((x+y)*z)$ ' (aceita)

Passo 1: $\delta(q_0, '(') \rightarrow q_0$, push '(' | pilha=['(', '\$']
 Passo 2: $\delta(q_0, '(') \rightarrow q_0$, push '(' | pilha=['(', '(', '\$']
 Passo 3: $\delta(q_0, 'x') \rightarrow q_0$, neutro
 Passo 4: $\delta(q_0, '+') \rightarrow q_0$, neutro
 Passo 5: $\delta(q_0, 'y') \rightarrow q_0$, neutro
 Passo 6: $\delta(q_0, ')') \rightarrow q_0$, pop '(' | pilha=['(', '\$']
 Passo 7: $\delta(q_0, '*') \rightarrow q_0$, neutro
 Passo 8: $\delta(q_0, 'z') \rightarrow q_0$, neutro
 Passo 9: $\delta(q_0, ')') \rightarrow q_0$, pop '(' | pilha=['\$'] → ACEITA

3.6 Execução Passo a Passo — ' $([])$ ' (rejeita)

Passo 1: $\delta(q_0, '(') \rightarrow q_0$, push '(' | pilha=['(', '\$']
 Passo 2: $\delta(q_0, '[') \rightarrow q_0$, push '[' | pilha=['[', '(', '\$']
 Passo 3: $\delta(q_0, ')') \rightarrow q_{DEAD}$ (topo='[', esperava '(') → REJEITA

4. Linguagem Recursiva — MT para $w#w$

4.1 Descrição

A linguagem reconhece cadeias da forma $w#w$, onde w é uma palavra binária (sobre $\{0,1\}$) e '#' é o separador. As duas cópias de w devem ser idênticas e ter o mesmo comprimento. Esta linguagem é **não livre de contexto** — não pode ser reconhecida por nenhum PDA — pois exige comparar substrings arbitrariamente longas posicionadas em regiões distintas da entrada.

Esta linguagem é o exemplo clássico que demonstra a necessidade de memória de trabalho irrestrita (fita da MT) e ilustra por que $LLC \neq R$ na hierarquia.

4.2 Definição Formal

$$\begin{aligned}\Sigma &= \{0, 1, \#\} \\ \Gamma &= \{0, 1, \#, X, Y, B\} \text{ (B = branco)} \\ L &= \{ w\#w \mid w \in \{0,1\}^* \}\end{aligned}$$

4.3 Modelo Computacional — Máquina de Turing

A MT possui 7 estados operacionais ($q_0, q_{1a}, q_{1b}, q_{2a}, q_{2b}, q_3, q_4$) mais os estados terminais q_A (aceita) e q_R (rejeita). Utiliza marcação com X (símbolo já processado à esquerda do $\#$) e Y (símbolo já processado à direita).

Estratégia: em cada rodada, a MT (1) marca o próximo símbolo não marcado à esquerda com X , (2) atravessa até encontrar o par correspondente à direita do $\#$, (3) verifica igualdade e marca com Y , (4) retorna ao início. Aceita quando todos os símbolos estão marcados; rejeita em qualquer divergência ou desbalanceamento de tamanho. Complexidade: $O(n^2)$ passos, onde $n = |w|$.

Tabela de Transição (δ) — MT $w\#w$:

Estado	Lê	Escreve	→ Estado	Dir	Papel
q_0	0	X	q_{1a}	R	Marca 0, carrega para comparar
q_0	1	X	q_{2a}	R	Marca 1, carrega para comparar
q_0	X	X	q_0	R	Pula X já marcado
q_0	#	#	q_4	R	Entra em verificação final
q_{1a}	0,1,X	=	q_{1a}	R	Atravessa lado esquerdo
q_{1a}	#	#	q_{1b}	R	Cruzou o separador
q_{1b}	Y	Y	q_{1b}	R	Pula Y já marcado
q_{1b}	0	Y	q_3	L	Par correto: 0=0, marca Y
q_{1b}	1	1	q_R	R	Divergência: esperava 0
q_{2b}	1	Y	q_3	L	Par correto: 1=1, marca Y
q_{2b}	0	0	q_R	R	Divergência: esperava 1
q_3	0,1,X,Y,#=		q_3	L	Retorna ao início
q_3	B	B	q_0	R	Reinicia rodada
q_4	Y	Y	q_4	R	Verifica lado direito marcado
q_4	B	B	q_A	R	Tudo marcado → ACEITA

q4	0,1	=	qR	R	Símbolo não marcado → REJEITA
----	-----	---	----	---	-------------------------------

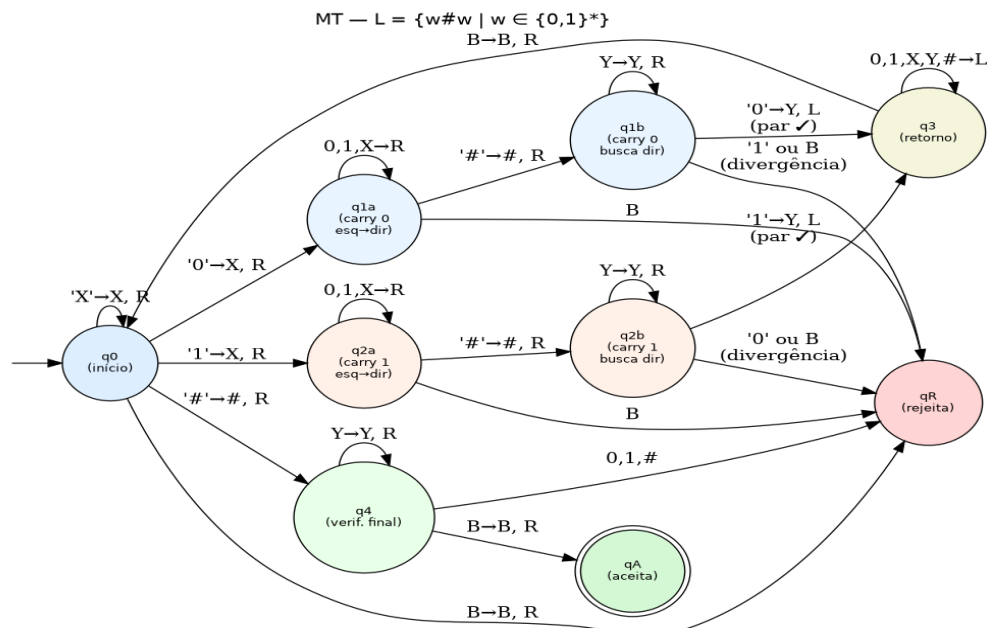


Figura 3 — Diagrama da Máquina de Turing para $w\#w$

4.4 Exemplos

Cadeia	Resultado	Passos
101#101	✓ ACEITA	44
0#0	✓ ACEITA	10
11001#11001	✓ ACEITA	102
101#100	✗ REJEITA (divergência no 3º símbolo)	29
101101	✗ REJEITA (sem separador #)	7
(vazio)	✗ REJEITA	1

4.5 Execução Passo a Passo — '101#101' (aceita, 44 passos)

Rodada 1 (carrega '1'):

$q_0, '1' \rightarrow (q_{2a}, 'X', R) \mid q_{2a}, '0' \rightarrow (q_{2a}, '0', R) \mid q_{2a}, '1' \rightarrow (q_{2a}, '1', R) \mid$
 $q_{2a}, '#' \rightarrow (q_{2b}, '#', R) \mid q_{2b}, '1' \rightarrow (q_3, 'Y', L) \mid q_3 \text{ volta ao início} \dots$

Rodada 2 (carrega '0'):

$q_0, 'X' \rightarrow (q_0, 'X', R) \mid q_0, '0' \rightarrow (q_{1a}, 'X', R) \mid \dots q_{1b}, '0' \rightarrow (q_3, 'Y', L) \mid \dots$

Rodada 3 (carrega '1'):

$q_0, 'X', 'X' \rightarrow \text{pula} \mid q_0, '1' \rightarrow (q_{2a}, 'X', R) \mid \dots q_{2b}, '1' \rightarrow (q_3, 'Y', L) \mid \dots$

Verificação (q4):

$q_0, '#' \rightarrow (q_4, '#', R) \mid q_4, 'Y' \rightarrow \dots \mid q_4, 'Y' \rightarrow \dots \mid q_4, 'Y' \rightarrow \dots \mid q_4, 'B' \rightarrow (q_A, 'B', R) \checkmark$

4.6 Execução Passo a Passo — '101#100' (rejeita, 29 passos)

Rodadas 1 e 2 funcionam normalmente (comparações '1'='1' e '0'='0' corretas).

Rodada 3 (carrega '1'):

$q_0, 'X', 'X' \rightarrow \text{pula} \mid q_0, '1' \rightarrow (q_{2a}, 'X', R) \mid q_{2a}, '#' \rightarrow (q_{2b}, '#', R) \mid$
 $q_{2b}, 'Y', 'Y' \rightarrow \text{pula} \mid q_{2b}, '0' - \text{esperava '1'} \rightarrow (q_R, '0', R) \rightarrow \text{REJEITA em 29 passos}$

5. Comparação e Conclusão

5.1 Resultados da Bateria de Testes

Nível	Modelo	Testes	Total Passos	Média	Complexidade
LR — CPF	DFA	9/9 (100%)	106	11,8	$O(n)$ — linear
LLC — Balanceamento	PDA	8/8 (100%)	43	5,4	$O(n)$ — linear
R — $w\#w$	MT	8/8 (100%)	227	28,4	$O(n^2)$ — quadrático

5.2 Análise da Hierarquia LR ■ LLC ■ R

Os três reconhecedores demonstram na prática a hierarquia de Chomsky:

DFA (CPF): reconhece linguagens regulares sem memória auxiliar. O número de estados é fixo e proporcional ao comprimento máximo da cadeia. Não consegue reconhecer linguagens como balanceamento (exigiria infinitos estados para rastrear profundidade arbitrária) nem $w\#w$.

PDA (Balanceamento): a pilha adiciona memória ilimitada de tipo LIFO, permitindo rastrear aninhamento arbitrário. É mais poderoso que o DFA, porém não consegue comparar duas substrings de posições arbitrárias na entrada — o que seria necessário para reconhecer $w\#w$.

MT ($w\#w$): a fita bidirecional com leitura e escrita permite comparação direta de substrings. A MT percorre a fita $O(n)$ vezes, resultando em $O(n^2)$ passos total. Esta linguagem não é livre de contexto (prova pelo lema do bombeamento), confirmando que $MT > PDA > DFA$ em poder expressivo.

5.3 Conclusão

O projeto implementou com sucesso três reconhecedores formais completos, cada um simulando explicitamente seu modelo computacional (estados, alfabeto, função de transição declarados como dados). Todos os 25 testes da bateria foram aprovados com 100% de acerto, e a contagem de passos confirma a diferença de complexidade prevista pela teoria: $O(n)$ para DFA e PDA, $O(n^2)$ para MT.

A hierarquia LR ■ LLC ■ R é assim demonstrada tanto teoricamente (pela definição formal de cada linguagem e pela impossibilidade de reconhecê-la com modelos menos poderosos) quanto empiricamente (pela diferença no número de passos e no tipo de memória utilizada).